

1. Tutorial: construcción de un libro de cálculo con el DSL

Este capítulo muestra, paso a paso, cómo construir un programa que genere automáticamente un libro de cálculo a partir de la descripción de un problema de horario. Al final del tutorial tendrás el archivo `Horario_Tesis.xlsx` en disco, listo para abrir.

1.1 Instalación

Instala los dos programas siguientes.

Steel Bank Common Lisp (SBCL) — descarga desde <https://www.sbcl.org/> e instala según las instrucciones de tu sistema operativo.

Python 3 con openpyxl:

```
pip install openpyxl
```

Descarga el generador. Los archivos que no debes modificar son la biblioteca del sistema. Tus dos archivos van en la **misma carpeta** que la biblioteca:

```
generador/  
+-- dsl-directo.lisp          <- biblioteca (no modificar)  
+-- ast-def.lisp             <- biblioteca (no modificar)  
+-- generate-code-direct.lisp <- biblioteca (no modificar)  
+-- code-generation-utils.lisp <- biblioteca (no modificar)  
+-- hoja_con_formulas.py     <- biblioteca (no modificar)  
+-- data-tutorial.lisp       <- tus datos (editar)  
\-- tutorial-horario.lisp     <- tu programa (editar)
```

No crees subcarpetas. El programa se ejecuta así desde esa misma carpeta:

```
cd generador/  
sbcl --load tutorial-horario.lisp
```

1.2 El problema

La Facultad de Matemática y Computación de la Universidad de La Habana celebra sus defensas de tesis en junio de 2026. Se tienen 13 defensas programadas en distintos locales (Postgrado, Salón decanato, Francofonía) durante los días 5, 8 y 10 de junio. Cada defensa tiene un estudiante, un tribunal de cinco profesores (tutor, presidente, secretario, vocal y oponente), un día, una hora y un local. Participan 32 profesores en total, algunos en más de una defensa. Se quiere saber cuántas defensas acumula cada profesor y detectar

automáticamente si algún profesor integra dos tribunales programados en el mismo día y hora.

El libro tendrá dos tablas en la misma hoja: una con las defensas y otra con los profesores.

1.3 Construyendo el programa paso a paso

Crea el archivo `tutorial-horario.lisp` en la carpeta del generador y ve añadiendo cada bloque según se describe a continuación.

1.3.1 Paso 1 — cargar el DSL

La primera línea del fichero carga el sistema:

Listing 1.1: `tutorial-horario.lisp` — inicio.

```
1 (load "dsl-directo.lisp")
```

`load` ejecuta el archivo indicado. `dsl-directo.lisp` carga a su vez el resto de la biblioteca (`ast-def`, `generate-code-direct`, etc.) y deja disponibles todas las formas del DSL: `def-table`, `libro`, `xl-generate`...

1.3.2 Paso 2 — preparar los datos

Crea el archivo `data-tutorial.lisp` en la misma carpeta. `defparameter` define una variable global con los datos de entrada. El formato es una lista de listas: la fila 1 y 2 son prefijos vacíos que el generador coloca antes de los datos, la fila 3 son los encabezados de columna, y a partir de la fila 4 van los datos reales. Las celdas que el generador va a calcular automáticamente se dejan como cadena vacía —el sistema las sobrescribirá con fórmulas.

Listing 1.2: `data-tutorial.lisp` (extracto).

```
1 (defparameter *DATOS-DEFENSAS*
2   '(((" " " " " " " " " " " " " ")
3     (" " " " " " " " " " " " ")
4     ("Dia" "Hora" "Estudiante" "Tutor"
5      "Presidente" "Secretario" "Vocal" "Oponente" "Local")
6     ;; --- 5 de junio ---
7     ("05/06/2026" "10:00"
8      "Angel Daniel Alonso Guevara"
9      "MSc. Carmen T. Fernandez Montoto"
10     "Dr. Alberto Fernandez Oliva"
11     "Lic. Alejandro Labourdette-Lantigua Soto"
12     "Lic. Dennis Daniel Gonzalez Duran"
13     "MSc. Aracelys Garcia Armenteros"
14     "Postgrado")
15     ;; --- 8 de junio ---
16     ("08/06/2026" "09:30"
17     "Adrian Hernandez Castellanos"
```

```

18     "Lic. Alejandra Monzon Pena"
19     "Lic. Amanda Noris Hernandez"
20     "Lic. Kevin Manzano Rodriguez"
21     "Lic. Daniel Abad Fundora"
22     "Lic. Rodrigo Garcia Gomez"
23     "Salon decanato")
24     ... ;; 11 filas mas en data-tutorial.lisp
25 ))
26
27 (defparameter *DATOS-PROFESORES*
28   '((" " " ")
29     (" " " ")
30     ("Nombre" "Defensas")
31     ("Dr. Alberto Fernandez Oliva" " ")
32     ("DrC. Alejandro Piad Morfis" " ")
33     ("Lic. Alejandra Monzon Pena" " ")
34     ("MSc. Fernando Raul Rodriguez Flores" " ")
35     ... ;; 28 profesores mas en data-tutorial.lisp
36 ))

```

El archivo completo con las 13 defensas y los 32 profesores está en `data-tutorial.lisp`. El nombre de cada profesor incluye su grado académico (MSc., Lic., DrC., etc.) para que la búsqueda que genera el sistema lo encuentre exactamente en las columnas de tribunal, donde los nombres también llevan grado.

Añade en `tutorial-horario.lisp` la línea que carga los datos:

Listing 1.3: `tutorial-horario.lisp` — cargar datos.

```

2 (load "data-tutorial.lisp")

```

A partir de este momento tienes disponibles las variables `*DATOS-DEFENSAS*` y `*DATOS-PROFESORES*`. Si añadieras una defensa más, bastaría con añadir una fila a `*DATOS-DEFENSAS*` —el resto del programa no cambia.

1.3.3 Paso 3 — declarar la tabla de defensas

Cada defensa es una fila. Las columnas siguen el orden del Excel fuente. Los cinco roles de tribunal van juntos y consecutivos para poder referenciarlos con un solo rango en las fórmulas.

Listing 1.4: `tutorial-horario.lisp` — tabla de defensas.

```

4 (def-table defensas-table
5   ((dia " ") (hora " ") (estudiante " ") (tutor " ")
6     (presidente " ") (secretario " ") (vocal " ") (oponente " ") (
7     local " "))
8   :cell-height 1
   :cell-width 1)

```

Qué hace `def-table`: Cada par (`nombre`) declara una columna. El primer elemento (`dia, tutor...`) es el *nombre interno*: el símbolo con el que te referirás a esa columna en

cualquier fórmula del resto del programa. El segundo elemento () es el valor por defecto de la celda; en esta tabla todos son vacíos porque los datos vienen de *DATOS-DEFENSAS*.

Si quisieras añadir una columna `observacion`, bastaría con incluir (`observacion`) en la lista y añadir el valor correspondiente en cada fila de *DATOS-DEFENSAS*.

1.3.4 Paso 4 — declarar la tabla de profesores

Cada profesor es una fila. La columna `defensas` se calcula automáticamente. El campo `nombre` almacena el nombre completo con grado ("MSc. Fernando Raúl Rodríguez Flores") para que coincida exactamente con los valores en las columnas de tribunal de `defensas-table`.

Listing 1.5: tutorial-horario.lisp — tabla de profesores.

```
10 (def-table profesores-table
11   ((nombre "") (defensas ""))
12   :cell-height 1
13   :cell-width 1
14   :computed
15     ((defensas (countif (trange defensas-table tutor oponente)
16                          (col nombre))))
17   :render
18     (conditional-rendering
19      :condition
20        (exists (r :rows-of defensas-table)
21         :self-in (tutor presidente secretario vocal oponente
22                   )
23         :matching (dia hora))
24     :target-columns (nombre)))
```

Qué hace :computed: Declara columnas cuyo valor es una fórmula, no un dato manual. La forma (`countif rango criterio`) genera una fórmula COUNTIF en Excel. (`trange defensas-table tutor oponente`) produce el rango que abarca desde la columna `tutor` hasta la columna `opponente` en todas las filas de `defensas-table`. (`col nombre`) devuelve el valor de la columna `nombre` en la fila actual de `profesores-table`. Combinados, la fórmula cuenta cuántas veces aparece este profesor en cualquiera de los cinco roles.

Modificación: contar por rol. Si quisieras saber además cuántas veces el profesor fue tutor específicamente, añadirías una columna `como-tutor`:

Listing 1.6: Columna adicional: solo tutores.

```
(def-table profesores-table
  ((nombre "") (defensas "") (como-tutor ""))
  ...
  :computed
    ((defensas (countif (trange defensas-table tutor
                                oponente)
                        (col nombre)))
     (como-tutor (countif (trange defensas-table tutor tutor)
                          (col nombre)))))
```

El segundo `countif` usa `(trange defensas-table tutor tutor)` —un rango de una sola columna— para contar solo apariciones en la columna `tutor`. Para contar como presidente usarías `(trange ... presidente presidente)`, y así con cada rol.

Qué hace `:render`: Asocia una condición de coloreado con una o más columnas destino. La condición usa `exists`: busca si existe alguna fila en `defensas-table` donde `:self-in` —los roles— contienen el nombre del profesor, y cuyo slot `dia + hora` aparece en más de una defensa simultánea (`:matching`). El sistema genera automáticamente una `FormulaRule` en el `.xlsx` que se reevalúa cada vez que el usuario edita el libro.

1.3.5 Paso 5 — ensamblar el libro

Las dos tablas van en la misma hoja, una al lado de la otra. Debajo de cada tabla se colocan etiquetas y totales.

Listing 1.7: `tutorial-horario.lisp` — workbook.

```

25 (libro horario-tesis
26   :filename "Horario_Tesis.xlsx"
27   :hojas (list
28     (hoja "Defensas"
29       (tabla defensas-table :data *DATOS-DEFENSAS*)
30       (tabla profesores-table :data *DATOS-PROFESORES*)
31       :fixed-expressions
32         (((nav (ultima-fila defensas-table dia) abajo 1)
33           (str "Total defensas:"))
34          ((nav (ultima-fila defensas-table dia) abajo 1 derecha
35              2)
36           (counta (trange defensas-table estudiante))))
37         (((nav (ultima-fila profesores-table nombre) abajo 1)
38           (str "Total profesores:"))
39          ((nav (ultima-fila profesores-table nombre) abajo 1
40              derecha 1)
41           (sum-range (trange profesores-table defensas))))))

```

Qué hace cada forma: `libro` crea el workbook con un nombre de archivo de salida. `hoja` define una pestaña; puede recibir más de una `tabla`, en cuyo caso las coloca en la misma hoja una al lado de otra. `tabla` instancia una plantilla (`def-table`) con datos concretos.

`:fixed-expressions` coloca fórmulas o etiquetas en celdas fuera de las tablas. La posición se expresa con `nav` y `ultima-fila`: `(nav (ultima-fila defensas-table dia) abajo 1)` calcula la celda que está exactamente una fila debajo de la última fila de datos de `defensas-table`, columna `dia`. `derecha 2` añade un desplazamiento de dos columnas a la derecha. Estas posiciones se recalculan automáticamente si el número de filas cambia, así que si añades más defensas los totales se desplazan solos.

1.3.6 Paso 6 — generar y ejecutar

Añade las dos últimas líneas al fichero:

Listing 1.8: tutorial-horario.lisp — generar y ejecutar.

```
44 (xl-generate horario-tesis "horario-tesis.py")
45 (xl-run-generated "horario-tesis.py")
```

`xl-generate` recorre el árbol AST que construyeron las macros anteriores y produce el script Python `horario-tesis.py`. `xl-run-generated` ejecuta ese script con Python 3 y escribe el archivo `Horario_Tesis.xlsx` en disco.

Ejecuta desde la carpeta del generador (la misma donde están todos los archivos `.lisp`):

```
sbcl --load tutorial-horario.lisp
```

1.4 Resultado

El archivo `Horario_Tesis.xlsx` aparece en la carpeta `mi-horario/`. Ábrelo con Excel o LibreOffice Calc.

Rango	Contenido
Columnas A–I, filas 4–16	<code>defensas-table</code> : 13 defensas de junio
Columna J	Separador en blanco
Columnas K–L, filas 4–35	<code>profesores-table</code> : 32 profesores
Columna L, filas 4–35	Fórmulas <code>COUNTIF</code> generadas
Fila 17, columnas A y C	Total de defensas con <code>COUNTA</code>
Fila 37, columnas K–L	Total de profesores con <code>SUM</code>
Rango K4:K35	<code>FormulaRule</code> : nombres en rojo si hay conflicto

Con este horario real, cada profesor aparece en entre 1 y 4 defensas (MSc. Fernando Raúl Rodríguez Flores y MSc. Aracelys García Armenteros participan en 4 cada uno). Los slots 10/06/2026 09:30 y 10/06/2026 13:30 tienen dos defensas simultáneas en distintos locales, pero los tribunales son completamente distintos, por lo que ningún profesor aparece en rojo: el sistema verifica correctamente que no hay conflictos en este horario.

1.5 El fichero completo

Listing 1.9: tutorial-horario.lisp completo.

```
1 (load "dsl-directo.lisp")
2 (load "data-tutorial.lisp")
3
4 ;; Tabla 1: una fila por defensa
5 ;; Columnas en el orden del Excel fuente.
6 ;; Los roles de tribunal son consecutivos (tutor...oponente)
   para
7 ;; referenciarlos con un unico trange en el COUNTIF y en el
   exists.
```

```

8 (def-table defensas-table
9   ((dia "") (hora "") (estudiante "") (tutor "")
10    (presidente "") (secretario "") (vocal "") (oponente "") (
11     local ""))
12   :cell-height 1
13   :cell-width 1)
14
15 ;; Tabla 2: una fila por profesor
16 ;; El nombre incluye el grado ("MSc. ...", "Lic. ...") para que
17 ;; COUNTIF lo encuentre tal cual en las columnas de tribunal.
18 (def-table profesores-table
19   ((nombre "") (defensas ""))
20   :cell-height 1
21   :cell-width 1
22   :computed
23     ((defensas (countif (trange defensas-table tutor oponente)
24                          (col nombre))))
25   :render
26     (conditional-rendering
27       :condition
28         (exists (r :rows-of defensas-table)
29           :self-in (tutor presidente secretario vocal oponente
30                     )
31           :matching (dia hora))
32       :target-columns (nombre)))
33
34 (libro horario-tesis
35   :filename "Horario_Tesis.xlsx"
36   :hojas (list
37     (hoja "Defensas"
38       (tabla defensas-table :data *DATOS-DEFENSAS*)
39       (tabla profesores-table :data *DATOS-PROFESORES*)
40       :fixed-expressions
41         (((nav (ultima-fila defensas-table dia) abajo 1)
42              (str "Total defensas:"))
43          ((nav (ultima-fila defensas-table dia) abajo 1 derecha
44                2)
45           (counta (trange defensas-table estudiante)))
46          ((nav (ultima-fila profesores-table nombre) abajo 1)
47           (str "Total profesores:"))
48          ((nav (ultima-fila profesores-table nombre) abajo 1
49                derecha 1)
50           (sum-range (trange profesores-table defensas))))))
51
52 (xl-generate horario-tesis "horario-tesis.py")
53 (xl-run-generated "horario-tesis.py")

```

1.6 Cómo funciona el sistema

Al cargar `tutorial-horario.lisp`, SBCL ejecuta las macros del DSL y construye en memoria un **árbol de sintaxis abstracta (AST)**. Cada nodo del árbol representa un concepto del problema: una tabla, una expresión de celda, una regla de coloreado.

`xl-generate` recorre ese árbol y produce un script Python. `xl-run-generated` ejecuta el script, que usa la biblioteca `openpyxl` para escribir el archivo `.xlsx`.

El DSL no contiene ninguna referencia al formato de salida: el mismo programa podría alimentar un generador diferente (PDF, HTML, LibreOffice) sin ningún cambio en el código del usuario.

1.7 Guía de las formas del DSL

1.7.1 *def-table* — *definir una tabla*

Sintaxis: `def-table`

```
1 (def-table nombre-tabla
2   ((col1 "Cabecera1") (col2 "Cabecera2") ...)
3   :cell-height N          ; filas físicas por fila lógica (
4     defecto 1)
5   :cell-width N           ; columnas físicas por columna
6     lógica (defecto 1)
7   :computed                ; columnas calculadas (opcional)
8     ((col-x (expresion)))
9   :render                  ; regla de coloreado (opcional)
10    (conditional-rendering ...))
```

Cada par `(columna )` declara el nombre interno del DSL y el texto del encabezado visual. La clave `:computed` asocia columnas a expresiones que el generador traduce a fórmulas. La clave `:render` acepta una forma `conditional-rendering` que especifica bajo qué condición se colorean determinadas columnas.

1.7.2 *exists* — *condición de existencia*

Sintaxis: `exists :rows-of`

```
1 (exists (var :rows-of tabla-id)
2   :self-in (col1 col2 ...) ; el valor actual aparece en
3     alguna de estas
4   :matching (clave1 clave2 ...)); ese slot tiene mas de una
5     fila programada
```

Sintaxis: exists :all-rows

```
1 (exists (var :all-rows)
2   :matching (col1 col2 ...) ; columnas que deben
   coincidir entre filas
3   :any-overlap (col1 col2 ...)) ; columnas donde algun valor
   debe compartirse
```

:rows-of busca en otra tabla; :all-rows busca en la misma tabla. El generador produce una fórmula SUMPRODUCT que persiste en el archivo y se reevalúa automáticamente cuando el usuario edita los datos.

1.7.3 nav — posicionamiento relativo

Sintaxis: nav / ultima-fila / primera-fila

```
1 (nav (ultima-fila tabla-id columna) abajo N)
2 (nav (ultima-fila tabla-id columna) abajo N derecha M)
3 (nav (primera-fila tabla-id columna) arriba N)
```

Calcula la posición de una celda como un ancla en una tabla más pasos direccionales (abajo, arriba, derecha, izquierda). Las coordenadas se recalculan cada vez que cambia el número de filas de datos, por lo que los totales de pie de tabla se desplazan solos.

1.7.4 libro / hoja / tabla — ensamblar el workbook

Sintaxis: libro / hoja / tabla

```
1 (libro nombre-workbook
2   :filename "nombre.xlsx"
3   :hojas (list
4     (hoja "Nombre hoja"
5       (tabla plantilla1 :data *DATOS-1*)
6       (tabla plantilla2 :data *DATOS-2*)
7       :fixed-expressions
8         (((nav ancla dir N) (expresion))
9         ...))))
```

Cuando una hoja recibe más de una tabla, el generador las coloca lado a lado separadas por una columna en blanco. :fixed-expressions coloca expresiones fuera de las tablas en posiciones relativas calculadas con nav.

1.8 Referencia rápida de expresiones

Forma	Descripción
(col nombre)	Valor de la columna en la fila actual
(trange tabla desde hasta)	Rango en otra tabla
(countif rango criterio)	Fórmula COUNTIF
(counta rango)	Fórmula COUNTA
(sum-range rango)	Fórmula SUM
(str "texto")	Literal de texto en celda
(_if cond entonces sino)	Condicional de tres ramas
(non-empty expr)	Verdadero si la expresión no está vacía
(show-nothing)	Produce la cadena vacía
(equals a b)	Igualdad
(different a b)	Desigualdad
(_and a b) / (_or a b)	Conjunción y disyunción
(add a b) / (subtract a b)	Suma y resta
(multiply a b) / (divide a b)	Multiplicación y división
(time-add hora minutos)	Suma minutos a una hora
(concat a b)	Concatenación de texto
(previous-row expr)	Valor en la fila anterior
(next-row expr)	Valor en la fila siguiente
(first-row)	Verdadero en la primera fila de datos
(tcol tabla nombre)	Columna de una tabla concreta (desambiguación)
(lookup clave campo)	Búsqueda estilo VLOOKUP
(xl-generate wb archivo)	Genera el script de construcción
(xl-run-generated archivo)	Ejecuta el script y crea el .xlsx

1.9 Formas adicionales del DSL

Plantillas paramétricas

Cuando la misma estructura de tabla se necesita varias veces con un detalle variable (por ejemplo, una tabla de franjas por cada día de la semana), se usan `:inst-params`:

```
1 (def-table franja-dia
2   ((turno "") (salon "") (grupo ""))
3   :inst-params (dia))
4
5 (hoja "Semana"
6   (tabla franja-dia :dia 'lunes :data *datos-lunes*)
7   (tabla franja-dia :dia 'martes :data *datos-martes*))
```

Celdas de varias filas o columnas

`:cell-height N` hace que cada fila lógica ocupe N filas físicas en la hoja generada.

```
1 (def-table horario-visual
2   ((hora "") (actividad "") (sala ""))
3   :cell-height 2)
```

Referencias cruzadas entre hojas

Forma	Para qué sirve
(sheet-ref hoja plantilla-celda)	Referencia a una celda de otra hoja con nombre fijo.
(cross-cell sheet xcol row)	Referencia a una celda de una hoja enlazada dinámicamente mediante <code>collect-over</code> .